OWASP ZAP

DAST

Dynamic Application Security Testing

LAPORAN PRAKTIKUM · DEVSECOPS PERTEMUAN 6

Pengujian Keamanan Aplikasi SnipVault

TARGET · VULNLAB.CYPHERA.MY.ID

TOOL · OWASP ZAP 2.17.0

KELOMPOK 6

Firmansyah Dzakwan Arifien — 0110223282
Azzam Fikri — 0110225110
Dhiya Nailah Rizqya — 0110225095
Ghifana Adzkia Ahmad — 0110225044
Rizky Putri — 0110225064
Muhammad Fathir Lubis — 0110225005
Muhammad Bintang Siregar — 0110225058

Teknik Informatika · STT Terpadu Nurul Fikri · 19 Oktober 2026

RINGKASAN EKSEKUTIF

Hasil Scan Otomatis OWASP ZAP

1	5	6	6
HIGH	MEDIUM	LOW	INFORMATIONAL

Scanner otomatis OWASP ZAP mengkonfirmasi **1 temuan High** — Cross-Site Scripting (Reflected) pada parameter `q` di /search.

CATATAN METODOLOGIS

DAST otomatis tidak menangkap kerentanan pada endpoint POST dan yang bergantung pada logika aplikasi. Pengujian manual terarah (Tugas E) menemukan empat temuan High tambahan: SSTI, IDOR, Sensitive Data Exposure, dan SSRF.

PROFIL TARGET

Aplikasi SnipVault

APLIKASI

Platform berbagi code snippet & catatan

URL

`vulnlab.cyphera.my.id`

TEKNOLOGI

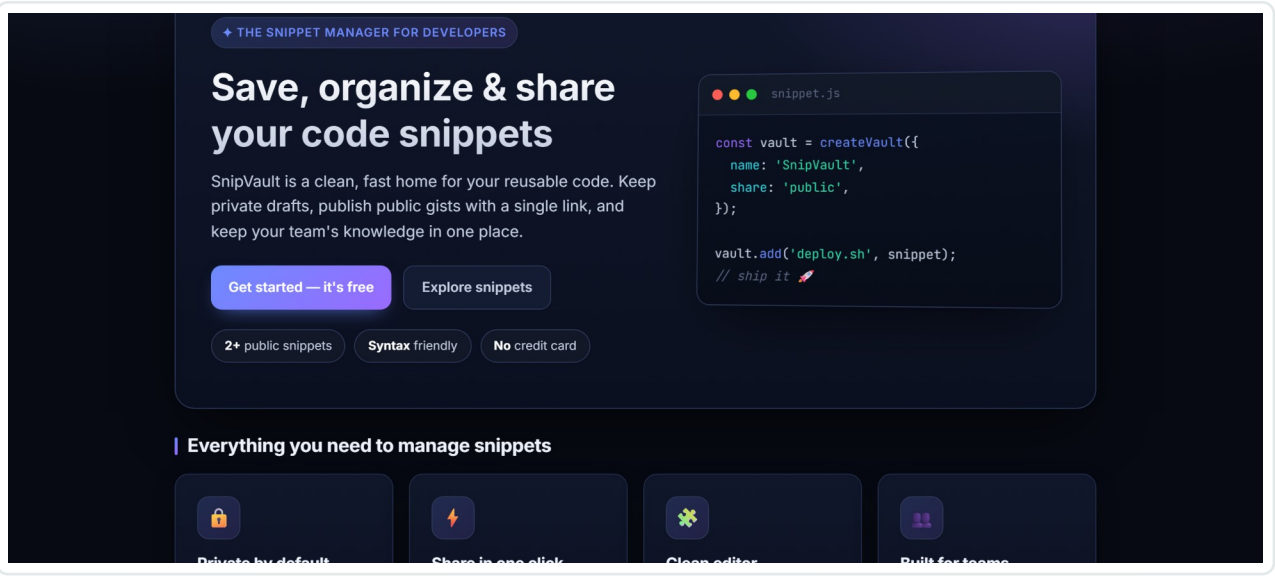
Node.js · Express · EJS (Vercel)

TOOL DAST

OWASP ZAP 2.17.0

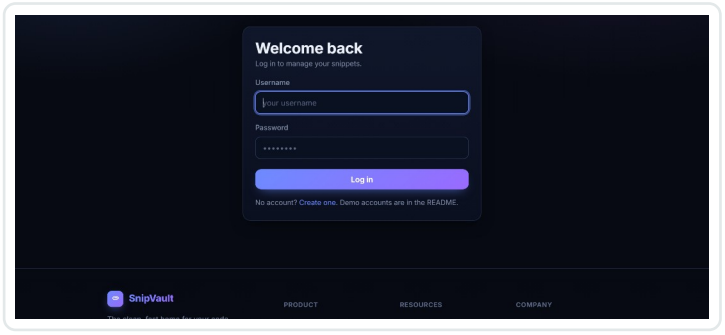
TANGGAL

Senin, 19 Oktober 2026

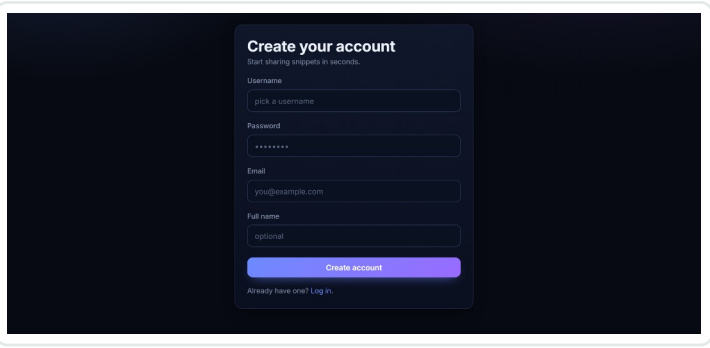


GBR 1 — Halaman utama aplikasi SnipVault

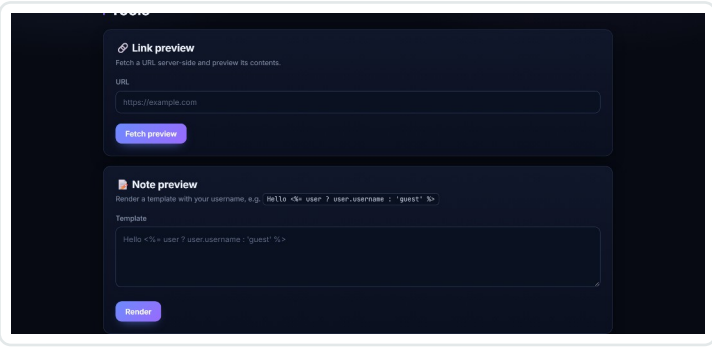
Tampilan & Fitur Aplikasi



GBR 2 — Halaman login



GBR 3 — Halaman registrasi



GBR 4 — /tools: Link & Note Preview

Permukaan serangan utama: /login & /register (SQLi, CSRF), /search (Reflected XSS), /tools → Link Preview (SSRF) & Note Preview (SSTI), /note/{id} (IDOR), /api/users (Sensitive Data Exposure).

METODOLOGI

Pendekatan Pengujian

01	**Traditional Spider**	Pemetaan struktur URL aplikasi secara otomatis
02	**AJAX Spider**	Crawling halaman yang dirender JavaScript
03	**Passive Scan**	Analisis traffic HTTP tanpa mengirim payload
04	**Active Scan**	Injeksi payload serangan untuk uji kerentanan
05	**Pengujian Manual**	IDOR, SSTI, SSRF — verifikasi logika bisnis (Tugas E)

Daftar Alert OWASP ZAP

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Cross Site Scripting (Reflected)	High	1
Absence of Anti-CSRF Tokens	Medium	Systemic
Content Security Policy (CSP) Header Not Set	Medium	Systemic
Cross-Domain Misconfiguration	Medium	Systemic
Missing Anti-clickjacking Header	Medium	Systemic
Sub Resource Integrity Attribute Missing	Medium	Systemic
Application Error Disclosure	Low	1
Cookie Without Secure Flag	Low	2
Cookie without SameSite Attribute	Low	2
Private IP Disclosure	Low	1
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Low	Systemic
X-Content-Type-Options Header Missing	Low	Systemic
Authentication Request Identified	Informational	2
Information Disclosure - Suspicious Comments	Informational	2
Re-examine Cache-control Directives	Informational	Systemic
Retrieved from Cache	Informational	Systemic
Session Management Response Identified	Informational	2
User Controllable HTML Element Attribute (Potential XSS)	Informational	4

GBR 5 — Daftar alert hasil Active Scan OWASP ZAP

RINGKASAN

High	1
Medium	5
Low	6
Informational	6

Konsisten dengan laporan HTML resmi ZAP (Lampiran A).

Laporan Resmi OWASP ZAP

ZAP by Checkmarx **DAST Report — SnipVault P6**

Kelompok 6 — DevSecOps Pertemuan 6. Anggota: Firmansyah Dzakwan Arifien, Azzam Fikri, Dhiya Nailah Rizqya, Ghifana Adzkia Ahmad, Rizky Putri, Muhammad Fathir Lubis, Muhammad Bintang Siregar.

Site: https://vulnlab.cyphera.my.id

Generated on Wed, 21 Oct 2026 16:05:23

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	1
Medium	5
Low	6
Informational	6
False Positives:	0

GBR 6 — Ringkasan laporan HTML ZAP (High 1 · Med 5 · Low 6 · Info 6)

High	Cross Site Scripting (Reflected)
Description	Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client, or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-supported technology. When an attacker gets a user's browser to execute his/her code, the code will run within the security context (or zone) of the hosting web site. With this level of privilege, the code has the ability to read, modify and transmit any sensitive data accessible by the browser. A Cross-site Scripted user could have his/her account hijacked (cookie theft), their browser redirected to another location, or possibly shown fraudulent content delivered by the web site they are visiting. Cross-site Scripting attacks essentially compromise the trust relationship between a user and the web site. Applications utilizing browser object instances which load content from the file system may execute code under the local machine zone allowing for system compromise. There are three types of Cross-site Scripting attacks: non-persistent, persistent and DOM-based. Non-persistent attacks and DOM-based attacks require a user to either visit a specially crafted link laced with malicious code, or visit a malicious web page containing a web form, which when posted to the vulnerable site, will mount the attack. Using a malicious form will oftentimes take place when the vulnerable resource only accepts HTTP POST requests. In such a case, the form can be submitted automatically, without the victim's knowledge (e.g. by using JavaScript). Upon clicking on the malicious link or submitting the malicious form, the XSS payload will get echoed back and will get interpreted by the user's browser and execute. Another technique to send almost arbitrary requests (GET and POST) is by using an embedded client, such as Adobe Flash. Persistent attacks occur when the malicious code is submitted to a web site where it's stored for a period of time. Examples of an attacker's favorite targets often include message board posts, web mail messages, and web chat software. The unsuspecting user is not required to interact with any additional site/link (e.g. an attacker site or a malicious link sent via email), just simply view the web page containing the code.
URL	https://vulnlab.cyphera.my.id/search?q=%3C%2Fp%3E%3Cscript%3Ealert%281%29%3B%3C%2FscRipt%3E%3Cp%3E
Node Name	https://vulnlab.cyphera.my.id/search (q)
Method	GET
Parameter	q
Attack	<scIpt>alert(1);</scRipt>
Evidence	<scIpt>alert(1);</scRipt>
Other Info	

GBR 7 — Detail alert High: Cross-Site Scripting (Reflected)

Laporan diekspor dari OWASP ZAP (Report → Generate Report, Traditional HTML Report) dan dilampirkan sebagai file HTML pada repositori kelompok.

TEMUAN HIGH — DIKONFIRMASI ZAP

Cross-Site Scripting (Reflected)

HIGH

ENDPOINT

`GET /search?q=`

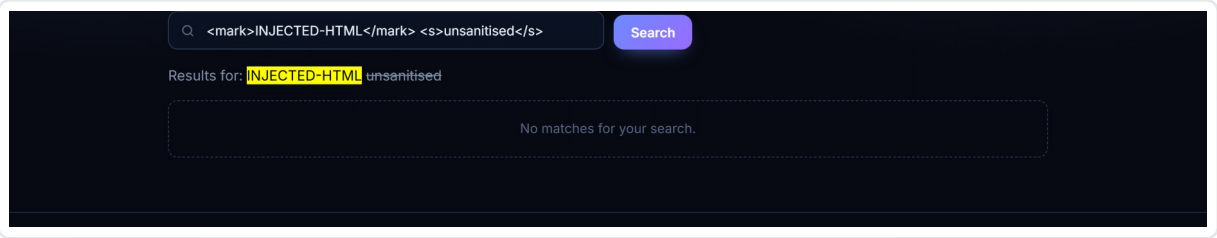
PARAMETER

`q`

PAYLOAD

```
</p><script>alert(1)</scRipt><p>
```

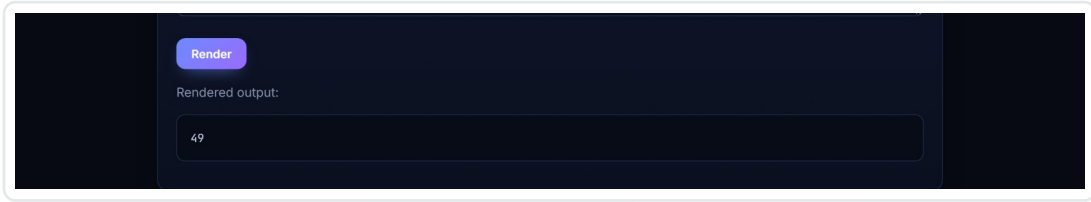
Input parameter q dipantulkan tanpa encoding sehingga tag HTML/JavaScript dieksekusi di browser korban — berpotensi pencurian session, defacement, dan phishing.



GBR 8 — Input pada /search?q= ter-render sebagai HTML

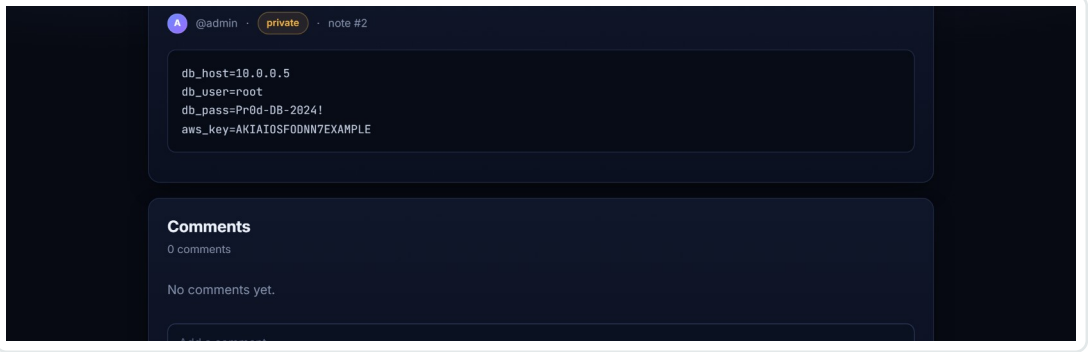
SSTI & IDOR

HIGH Server-Side Template Injection — /preview



Payload <%= 7*7 %> dievaluasi server menjadi 49 — berpotensi Remote Code Execution.

HIGH IDOR / Broken Access Control — /note/{id}



Mengganti id membuka private note milik pengguna lain; note admin membocorkan kredensial DB & AWS key.

Sensitive Data Exposure & SSRF

HIGH

Sensitive Data Exposure — GET /api/users



GBR 9 — Endpoint publik membocorkan hash password & api_key seluruh pengguna

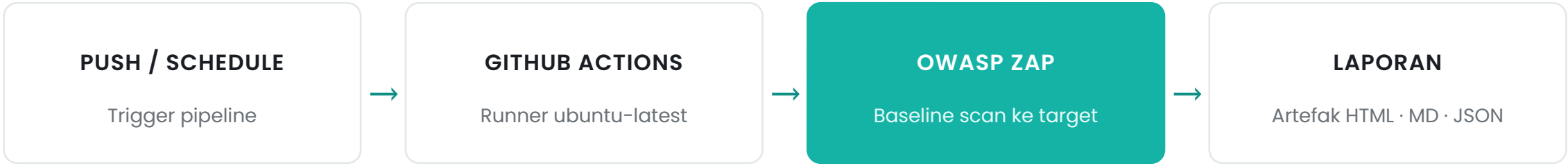
HIGH

SSRF — Server-Side Request Forgery · GET /fetch?url=

Fitur Link Preview memfetch URL arbitrer di sisi server tanpa validasi. Saat diisi alamat internal — `http://localhost/` atau metadata `http://169.254.169.254/` — server tetap memprosesnya, membuka jalan pivot ke jaringan internal.

INTEGRASI DEVSECOPS

DAST Otomatis via GitHub Actions + OWASP ZAP



.github/workflows/dast-zap.yml

```yaml
name: DAST - OWASP ZAP Baseline Scan
on:
  workflow_dispatch:
  schedule: [ { cron: '0 2 * * 1' } ]
jobs:
  zap_baseline:
    runs-on: ubuntu-latest
    steps:
      - uses: zaproxy/action-baseline@v0.12.0
        with:
          target: 'https://vulnlab.cyphera.my.id'
          fail_action: false
```

REKOMENDASI REMEDIASI

Prioritas High & Medium

KERENTANAN	SEV	REMEDIASI
Cross-Site Scripting (XSS)	HIGH	Output encoding kontekstual (HTML-escape) + Content-Security-Policy ketat
SSTI	HIGH	Jangan render input sebagai template; pisahkan template dari data
IDOR / Broken Access Control	HIGH	Authorization check kepemilikan objek pada setiap request
Sensitive Data Exposure	HIGH	Autentikasi endpoint; jangan kembalikan field sensitif; rotasi kredensial
SSRF	HIGH	Allow-list domain; blokir IP privat/loopback/link-local
Header & Cookie	MEDIUM	Anti-CSRF token, CSP, X-Frame-Options, nosniff, Secure/HttpOnly/SameSite

KESIMPULAN

Intisari & Pembelajaran

01 **Scan otomatis menemukan 18 jenis alert**

1 High (Reflected XSS), 5 Medium, 6 Low, 6 Informational.

02 **Pengujian manual melengkapi scanner**

Empat temuan High tambahan: SSTI, IDOR, Sensitive Data Exposure, SSRF.

03 **Otomatis & manual saling melengkapi**

Active Scan untuk parameter GET; manual untuk endpoint POST & logika bisnis.

04 **Integrasi CI/CD**

OWASP ZAP dijalankan otomatis via GitHub Actions sebagai gerbang DevSecOps.

Repositori `github.com/FirmansyahDzakwanArifien/DevSecOPS · praktikum/dast-p6`

Laporan HTML `Kelompok-6_DAST_P6/ZAP_DAST_Report_SnipVault_P6.html`